

Aula 16 — SharedPreferences

Desenvolvimento para Ambientes Móveis · Faculdade COTEMIG | 2026.2

FLUTTER PERSISTÊNCIA

AULA 16



Por que estamos aqui?

Você já percebeu que, ao fechar o app, tudo que o usuário digitou ou configurou simplesmente **some**? Isso acontece porque `setState` só mantém dados na memória e memória é volátil.

O problema

O `setState` desaparece ao encerrar o app. Preferências, favoritos e último login são perdidos.

O que o Capstone exige

Seu projeto final precisa gravar favoritos/último usuário logado/preferências de tema (Utilizar o `SharedPreferences`)

A solução de hoje

Aprender a **gravar dados no disco** do celular usando `SharedPreferences`.



Memória × Disco

Entender a diferença entre memória e disco é fundamental para escolher a ferramenta certa em cada situação.

Memória — `setState`

- Acesso extremamente rápido
- Dados vivem apenas enquanto o app está aberto
- Ao fechar ou reiniciar: tudo se perde
- Ideal para estado temporário da UI

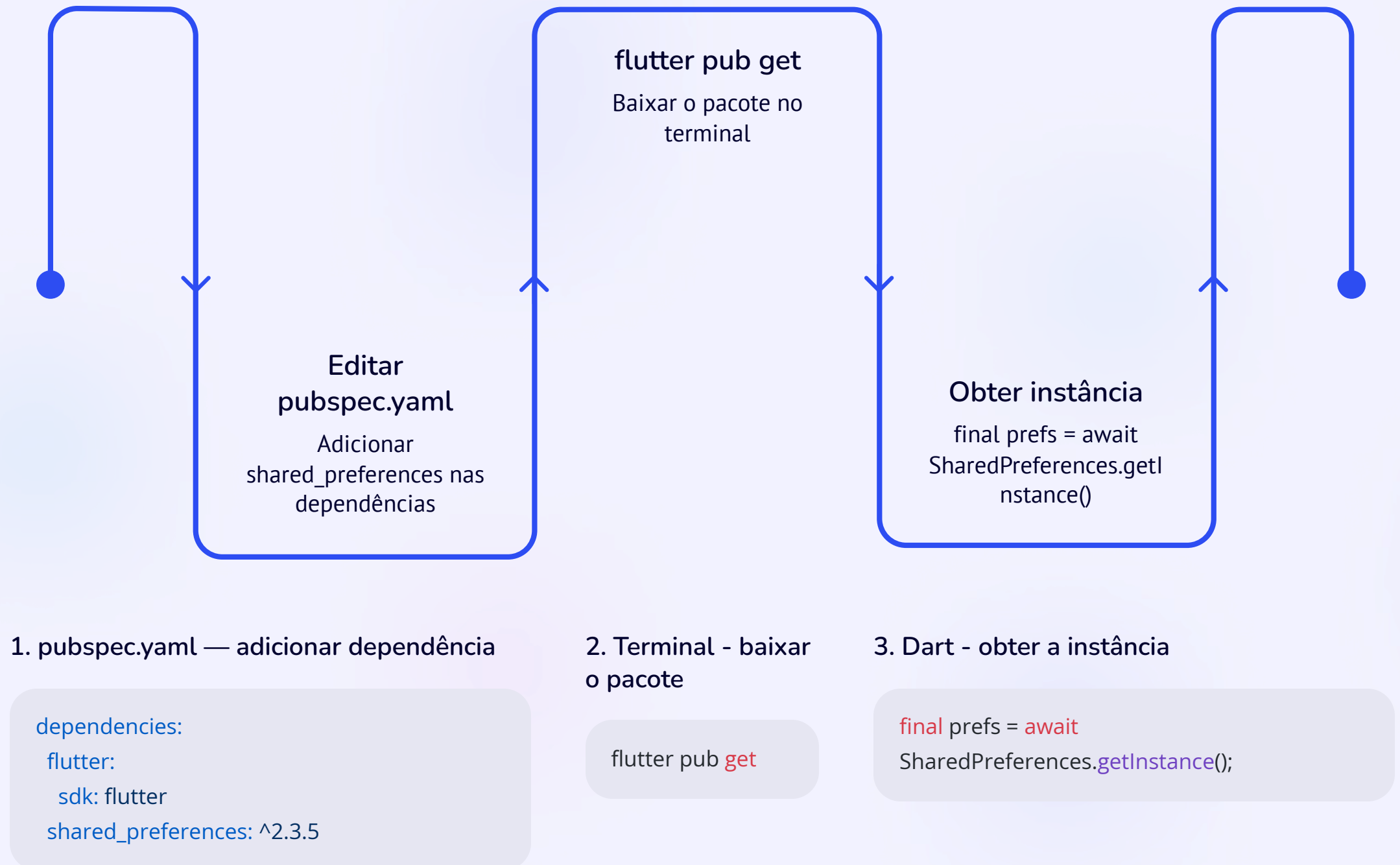
Disco — `SharedPreferences`

- Leitura/escrita assíncrona
- Dados **persistem** entre sessões do app
- Armazenamento key/value simples
- **Não é banco SQL** — sem consultas complexas

 Use `SharedPreferences` para dados leves e simples. Para dados relacionais ou grandes volumes, considere SQLite ou Hive.

Adicionando o Package

Antes de usar o `SharedPreferences`, é necessário adicionar o pacote ao projeto e importá-lo. O processo é simples e segue o padrão Flutter.



❏ Após editar o `pubspec.yaml`, execute `flutter pub get` no terminal para baixar o pacote. O método `getInstance()` é **assíncrono** — sempre use `await` e garanta que a função que o chama seja marcada como `async`.

API Essencial

O `SharedPreferences` oferece métodos para salvar e recuperar tipos primitivos. Cada tipo tem seu par de métodos – um para gravar (`set`) e outro para ler (`get`).

String

```
setString('chave', valor)
getString('chave')
```

Bool

```
setBool('chave', valor)
getBool('chave')
```

Int

```
setInt('chave', valor)
getInt('chave')
```

Double

```
setDouble('chave', valor)
getDouble('chave')
```

List<String>

```
setStringList('chave', lista)
getStringList('chave')
```

⚠ E se eu precisar de um Map?

`SharedPreferences` **não suporta** `Map` diretamente.

A solução é converter o `Map` para JSON usando `dart:convert` e salvar como `String`:

- Use `jsonEncode` para serializar e `jsonDecode` para desserializar.
- Para `Maps` complexos ou grandes volumes de dados, considere usar **Hive** ou **SQLite**.

```
import 'dart:convert';
```

```
// Salvar
```

```
final json = jsonEncode(meuMap);
await prefs.setString('meuMap', json);
```

```
// Carregar
```

```
final raw = prefs.getString('meuMap');
final map = raw != null
  ? jsonDecode(raw) as Map<String, dynamic>
  : {};
```

⚠ Todos os métodos `get` retornam `null` se a chave não existir – use sempre `?? valorPadrão` para fornecer um fallback seguro.

🗑 Remover uma chave

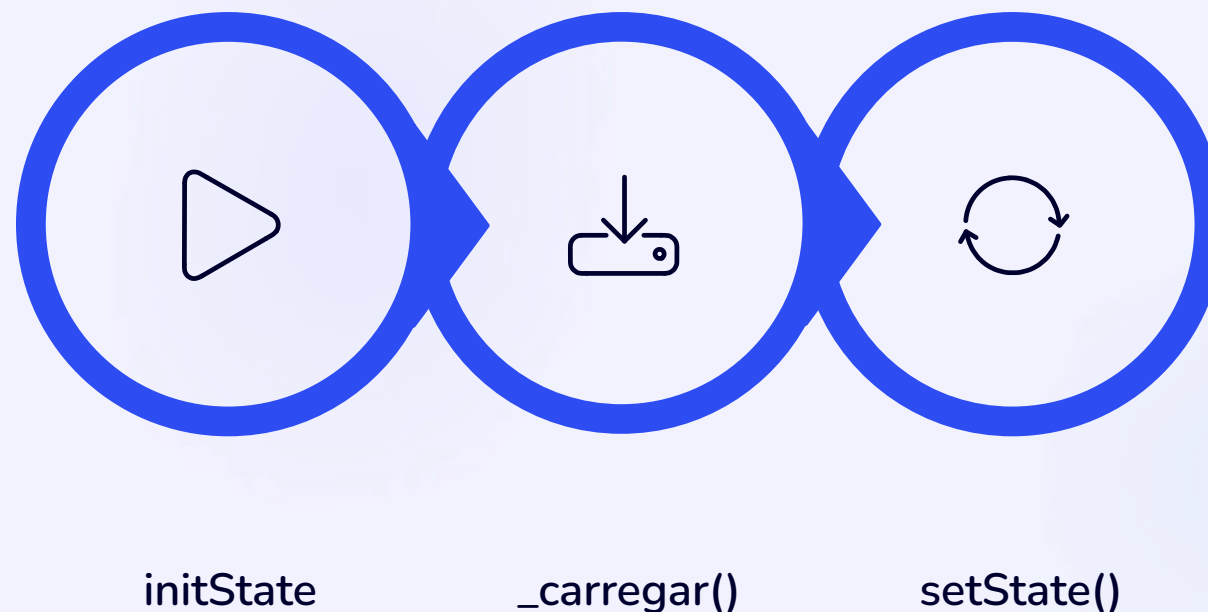
```
prefs.remove('chave');
```

🧹 Limpar tudo

```
prefs.clear();
```

Carregando Dados no initState

O `initState` é o lugar certo para carregar dados ao iniciar um widget – mas ele **não pode ser assíncrono**. A solução é delegar a lógica para um método separado.



```
@override
void initState() {
  super.initState();
  _carregar(); // chama sem await
}

Future<void> _carregar() async {
  final prefs = await SharedPreferences.getInstance();
  setState(() {
    _nome = prefs.getString('nome') ?? '';
    _temEscuro = prefs.getBool('temaEscuro') ?? false;
  });
}
```

⚠ Nunca marque o próprio `initState` como `async`. Isso causa erros em tempo de execução no Flutter.

Casos de Uso no Capstone

Veja como aplicar o `SharedPreferences` diretamente nos requisitos do seu projeto final:



Último usuário

Salvar o nome ou e-mail no `TextField` de login com `setString` para preencher automaticamente na próxima vez.



Favorito por ID

Gravar um `bool` com a chave sendo o ID do item: `setBool('fav_42', true)` para rastrear favoritos individualmente.



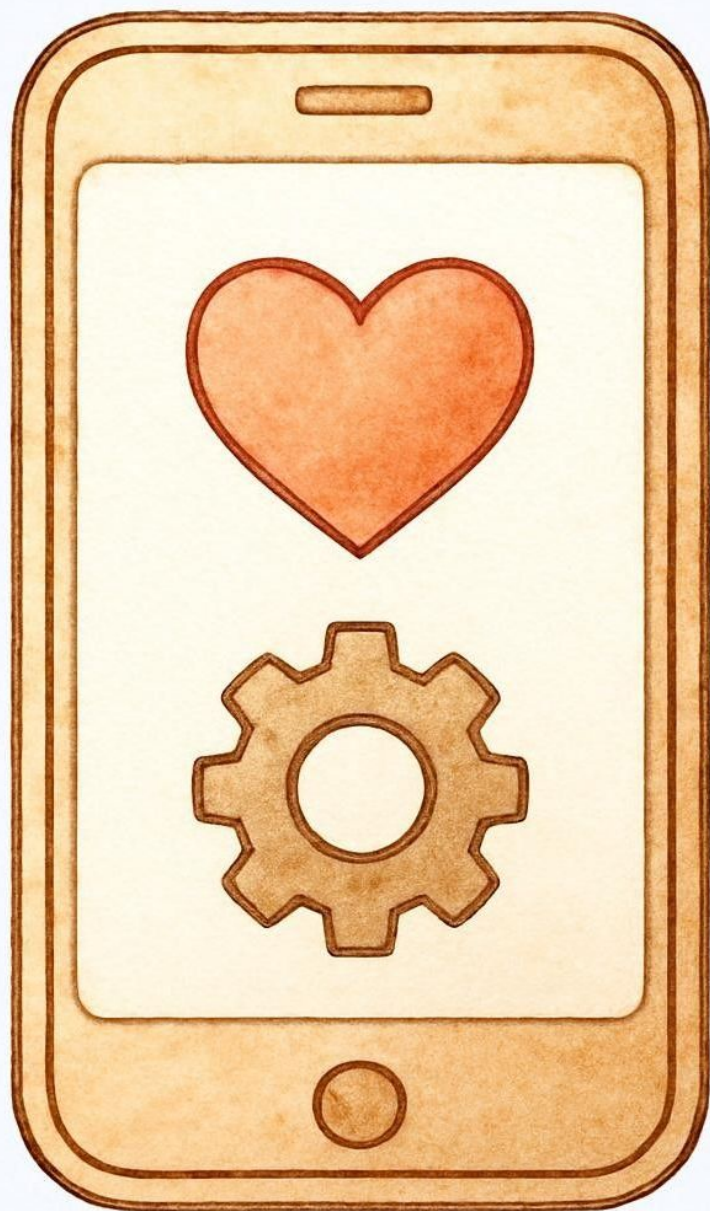
Lista de IDs

Usar `setStringList` para salvar todos os IDs favoritados de uma vez, ideal para exibir a tela de favoritos.



Tema escuro

Um simples `setBool('temaEscuro', true)` já é suficiente para persistir a preferência visual do usuário.



Demo: Vendo a Persistência em Ação

A melhor forma de entender a diferença entre memória e disco é comparando o comportamento dos dois lado a lado.

✓ Com SharedPreferences

1. Digite um valor e salve com `setString`
2. Faça um **Hot Restart** no VS Code
3. O valor **volta automaticamente** do disco
4. Mesmo após fechar o app completamente

✗ Só com setState

1. Digite um valor e atualize o estado
2. Faça um **Hot Restart** no VS Code
3. O valor **desaparece** – memória foi limpa
4. O usuário precisa digitar tudo de novo

✓ Hot Restart reinicia o app completamente, simulando o comportamento real de fechar e abrir o aplicativo.



Lab - Mão na Massa

Coloque em prática tudo que foi visto hoje. Siga os passos abaixo para criar um mini-app que persiste nome e tema do usuário:

1

Carregar nome e tema

No `initState`, chame um método `_carregar()` `async` que lê uma `String` (nome) e um `bool` (tema escuro) das preferências.

2

Salvar `String` e `bool`

Crie botões que chamam `setString` e `setBool` para persistir os dados no disco ao clicar.

3

Provar com Hot Restart

Salve os dados, faça um Hot Restart e confirme que o nome e o tema voltam exatamente como foram gravados.

Próxima Aula - Provider

Hoje você aprendeu a **persistir dados no disco**. Na próxima aula, vamos um nível acima: gerenciar o estado entre múltiplas telas do app.

SharedPreferences — hoje

- Salva dados **no disco** do celular
- Persiste entre sessões e reinicializações
- Ideal para preferências e configurações simples
- Acesso via `await`, escopo local do widget

Provider — 29/09

- Gerencia **estado entre telas**
- Dados disponíveis globalmente na árvore de widgets
- Complementa o SharedPreferences
- Base para arquiteturas como MVC e MVVM em Flutter

 **Dica:** Os dois se complementam – use `SharedPreferences` para persistir no disco e `Provider` para compartilhar o estado carregado entre as telas.